Національний технічний університет України
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Радіотехнічний факультет

Кафедра радіотехнічних систем

ЗВІТ

до лабораторної роботи № 1

Частина 2. «Розрахунок таблиці функції»

з дисципліни «Інформатика. Частина 1.
Основи програмування та алгоритми»

Виконав: студент групи РІ-61

Карацюба Олександр

Перевірив:

Київ — 2026

1. Мета роботи

Удосконалення знань, умінь та навичок розроблення програмного забезпечення мовою С у процедурній парадигмі; здобуття практичних навичок організації текстового інтерфейсу з валідацією введених даних, використання операторів розгалуження та циклів (do…while, for), виведення даних у вигляді таблиці з посторінковою зупинкою; закріплення навички побудови блок-схем алгоритмів із розгалуженнями та циклами. Метою другої частини лабораторної роботи № 1 є розроблення програми, яка розраховує таблицю значень функції f(x) відповідно до варіанта № 16 та визначає проміжки зміни знака функції (проміжки ізоляції коренів нелінійного рівняння f(x) = 0) для підготовки до лабораторної роботи № 3.

2. Завдання

Розробити мовою програмування С консольну програму, яка будує таблицю значень функції варіанта № 16:

$$f(x) = (x - 50)\cdot\cos(x / 10) - 3x + 500$$

Ширини стовпців таблиці (за таблицею 4 методички для варіанта № 16): «номер точки» — 11, «аргумент» — 22, «значення функції» — 26 символів. Програма повинна:

1) надавати вибір одного з двох варіантів введення початкових даних: перший — початкове значення аргументу X1, кінцеве значення X2 та кількість точок N (крок delta = (X2 − X1) / (N − 1)); другий — X1, X2 та крок delta (кількість точок N = ⌊(X2 − X1) / delta⌋ + 1);

2) зберігати X1, X2, delta у змінних типу double, а N — у змінній типу unsigned int;

3) виконувати валідацію номера варіанта введення та початкових даних;

4) перед виведенням таблиці виводити введені початкові дані, малювати таблицю символами псевдографіки та зупиняти виведення кожні 10 рядків до натискання клавіші ENTER;

5) визначати та виводити проміжки, на яких функція змінює знак ($f(x_{i-1})\cdot f(x_i) \leq 0$), тобто проміжки ізоляції коренів. Побудувати блок-схему алгоритму. Вихідний код розмістити на сайті https://github.com/. Звіт до другої частини лабораторної роботи № 1 оформити відповідно до вказаних вимог.

3. Блок-схема алгоритму

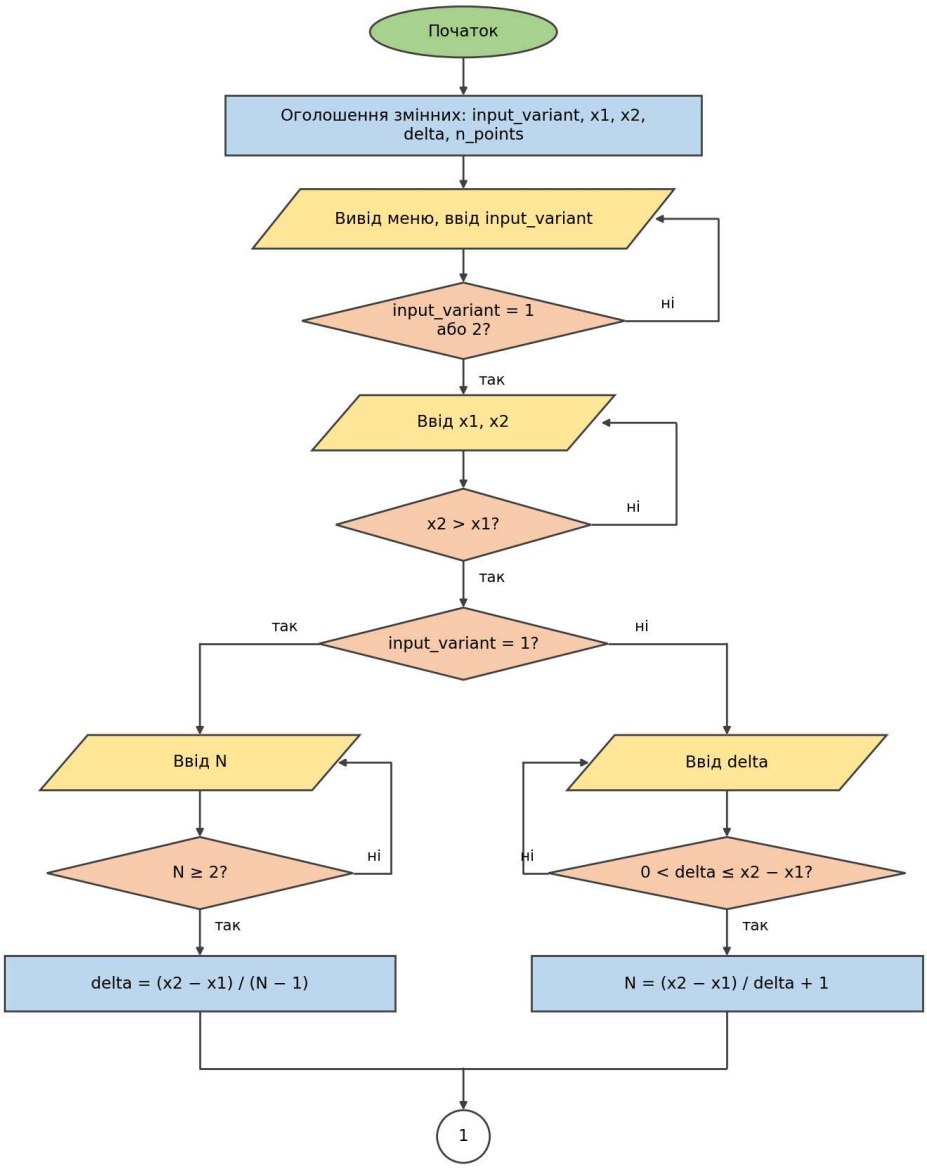


Рисунок 1 — Блок-схема алгоритму. Частина 1: вибір варіанта введення, введення та валідація X1, X2, N або delta

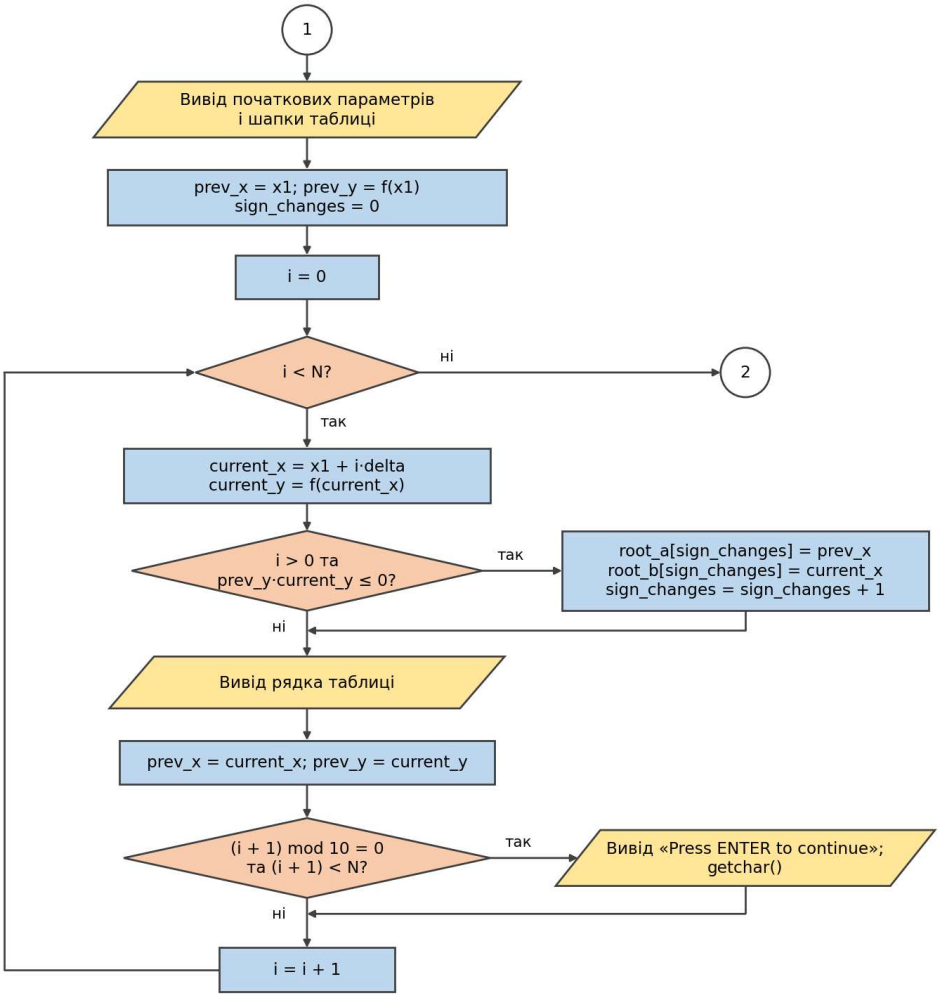


Рисунок 2 — Блок-схема алгоритму. Частина 2: цикл розрахунку й виведення таблиці, пошук проміжків зміни знака

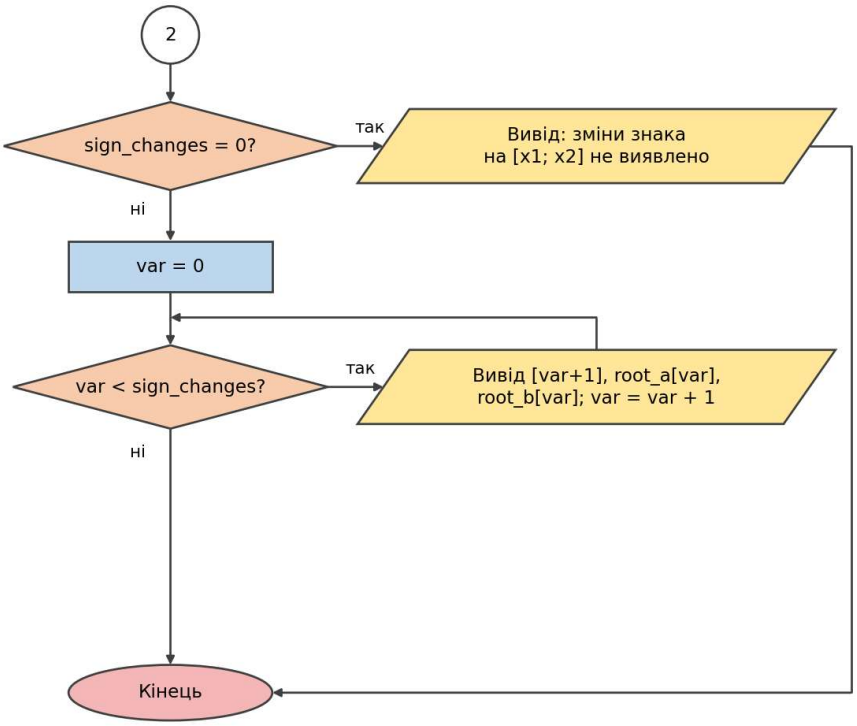


Рисунок 3 — Блок-схема алгоритму. Частина 3: виведення знайдених проміжків ізоляції

4. Текст програми

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

double f(double x)
{
    return (x-50.) * cos(x / 10) - 3. * x + 500.;
}

void print_border_line(void)
{
    printf("+");
    for (int i = 0; i < 11; i++) printf("-");
    printf("+");
    for (int i = 0; i < 22; i++) printf("-");
    printf("+");
    for (int i = 0; i < 26; i++) printf("-");
    printf("+\n");
}

int main()
{
    int input_variant = 0;
    double x1 = 0.0;
    double x2 = 0.0;
    double delta = 0.0;
    unsigned int n_points = 0;

    // Вибір варіанта введення
    do {
        printf("=====================================================\n");
        printf("  LABORATORY WORK #1. VARIANT #16\n");
        printf("  Function: f(x) = (x - 50)*cos(x/10) - 3*x + 500\n");
        printf("=====================================================\n");
        printf("Select input method:\n");
        printf("  1 - Enter X1, X2 and number of points N\n");
        printf("  2 - Enter X1, X2 and step delta\n");
        printf("Your choice (1 or 2): ");
        scanf("%d", &input_variant);

        if (input_variant != 1 && input_variant != 2) {
            printf("\n[ERROR] Invalid option! Please enter 1 or 2.\n\n");
        }
    } while (input_variant != 1 && input_variant != 2);

    // Введення меж
    do {
        printf("\nEnter start argument x1: ");
        scanf("%lf", &x1);
        printf("\nEnter start argument x2 (must be greater than X1): ");
        scanf("%lf", &x2);

        if (x2 <= x1)
        {
            printf("[ERROR] X2 must be strictly greater than X1!\n");
        }
    } while (x2 <= x1);

    // Розрахунок параметрів
    if (input_variant == 1)
    {
        do {
            printf("Enter number of points N (N => 2): ");
            scanf("%u", &n_points);
            if (n_points < 2)
            {
                printf("[ERROR] N must be a least 2! \n");
            }
        } while (n_points < 2);

        delta = (x2 - x1) / (double)(n_points - 1);
    }
    else {
        do {
            printf("Enter step delta (delta > 0): ");
            scanf("%lf", &delta);
            if (delta <= 0. || delta > (x2 - x1))
            {
                printf("[ERROR] Delta must be > 0 and <= (X2 - X1)!\n");
            }
```

```
        } while (delta <= 0. || delta > (x2 - x1));

        n_points = (unsigned int)((x2 - x1) / delta) + 1;
    }

    printf("\nInitial parameter:\n");
    printf("X1 = %.4lf, X2 = %.4lf, delta = %.4lf, N = %u\n\n", x1, x2, delta, n_points);

    //Друк таблиці
    print_border_line();
    printf("|  Point N  |     Argument (X)     |     Function (F(X))      |\n");
    print_border_line();

    getchar();

    int page_size = 10;
    double prev_x = x1;
    double prev_y = f(x1);

    double root_a[100];
    double root_b[100];
    int sign_changes = 0;

    for (unsigned int i = 0; i < n_points; i++)
    {
        double current_x = x1 + i * delta;
        double current_y = f(current_x);

        // Перевірка зміни знака
        if (i > 0)
        {
            if (prev_y * current_y <= 0.)
            {
                root_a[sign_changes] = prev_x;
                root_b[sign_changes] = current_x;
                sign_changes++;
            }
        }

        // Виведення рядка
        printf("|%10u |%21.6lf |%25.6lf |\n", i + 1, current_x, current_y);
        print_border_line();

        prev_x = current_x;
        prev_y = current_y;

        // Посторінкове виведення
        if ((i + 1) % page_size == 0 && (i + 1) < n_points)
        {
            printf("Press ENTER to continue");
            getchar();
        }
    }

    // Виведення результатів пошуку коренів
    printf("\n============================================================\n");
    printf("  Root isolation intervals:\n");
    printf("============================================================\n");
    if (sign_changes == 0)
    {
        printf("  No sign change detected on the interval [%.2lf; %.2lf].\n", x1, x2);
    }
    else {
        for (int var = 0; var < sign_changes; var++)
        {
            printf("  [%d] Root is between X = %.2lf and X = %.6lf\n", var + 1, root_a[var], root_b[var]);
        }
    }
    printf("============================================================\n");

    return 0;
}
```

5. Результати тестування

Програму скомпільовано компілятором GCC 13.3.0 без попереджень (прапорці -Wall -Wextra) і протестовано в консольному режимі.

```
gcc -Wall -Wextra main.c -o lab1 -lm
```

Тест 1 (варіант введення 2: X1, X2, delta; таблиця з 16 точок, що виводиться у два етапи). Після 10-го рядка виведення зупиняється до натискання ENTER.

```
==================================================
  LABORATORY WORK #1. VARIANT #16
  Function: f(x) = (x - 50)*cos(x/10) - 3*x + 500
==================================================
Select input method:
  1 - Enter X1, X2 and number of points N
  2 - Enter X1, X2 and step delta
Your choice (1 or 2): 2


Enter start argument x1: 0


Enter start argument x2 (must be greater than X1): 300
Enter step delta (delta > 0): 20


Initial parameter:
X1 = 0.0000, X2 = 300.0000, delta = 20.0000, N = 16


+-----------+----------------------+--------------------------+
|  Point N  |     Argument (X)     |     Function (F(X))      |
+-----------+----------------------+--------------------------+
|         1 |             0.000000 |               450.000000 |
+-----------+----------------------+--------------------------+
|         2 |            20.000000 |               452.484405 |
+-----------+----------------------+--------------------------+
|         3 |            40.000000 |               386.536436 |
+-----------+----------------------+--------------------------+
|         4 |            60.000000 |               329.601703 |
+-----------+----------------------+--------------------------+
|         5 |            80.000000 |               255.634999 |
+-----------+----------------------+--------------------------+
|         6 |           100.000000 |               158.046424 |
+-----------+----------------------+--------------------------+
|         7 |           120.000000 |               199.069777 |
+-----------+----------------------+--------------------------+
|         8 |           140.000000 |                92.306350 |
+-----------+----------------------+--------------------------+
|         9 |           160.000000 |               -85.342543 |
+-----------+----------------------+--------------------------+
|        10 |           180.000000 |                45.841172 |
+-----------+----------------------+--------------------------+
Press ENTER to continue
|        11 |           200.000000 |               -38.787691 |
+-----------+----------------------+--------------------------+
|        12 |           220.000000 |              -329.993340 |
+-----------+----------------------+--------------------------+
|        13 |           240.000000 |              -139.405989 |
+-----------+----------------------+--------------------------+
|        14 |           260.000000 |              -144.146942 |
+-----------+----------------------+--------------------------+
|        15 |           280.000000 |              -561.399349 |
+-----------+----------------------+--------------------------+
|        16 |           300.000000 |              -361.437138 |
+-----------+----------------------+--------------------------+


==========================================================
  Root isolation intervals:
==========================================================
  [1] Root is between X = 140.00 and X = 160.000000
  [2] Root is between X = 160.00 and X = 180.000000
  [3] Root is between X = 180.00 and X = 200.000000
==========================================================
```

Перевірка: значення функції в окремих точках розраховано вручну.

f(100) = (100 − 50)·cos(10) − 3·100 + 500 = 50·(−0,83907153) + 200 = −41,953577 + 200 = 158,046424.

f(160) = (160 − 50)·cos(16) − 3·160 + 500 = 110·(−0,95765948) + 20 = −105,342543 + 20 = −85,342543.

f(180) = (180 − 50)·cos(18) − 3·180 + 500 = 130·0,66031671 − 40 = 85,841172 − 40 = 45,841172.

Аргументи тригонометричних функцій — у радіанах. Усі три значення збігаються з виведеними програмою (точки 6, 9 і 10). Знак функції змінюється між точками 8 і 9 (92,31 → −85,34), 9 і 10 (−85,34 → 45,84) та 10 і 11 (45,84 → −38,79), тому програма знайшла три проміжки: [140; 160], [160; 180] та [180; 200].

Тест 2 (варіант введення 1: X1, X2, N). N = 10, тому крок delta = (200 − 140) / (10 − 1) = 6,6667; таблиця виводиться на одному екрані без зупинки.

```
Your choice (1 or 2): 1


Enter start argument x1: 140


Enter start argument x2 (must be greater than X1): 200
Enter number of points N (N => 2): 10


Initial parameter:
X1 = 140.0000, X2 = 200.0000, delta = 6.6667, N = 10


+-----------+----------------------+--------------------------+
|  Point N  |     Argument (X)     |     Function (F(X))      |
+-----------+----------------------+--------------------------+
|         1 |           140.000000 |                92.306350 |
+-----------+----------------------+--------------------------+
|         2 |           146.666667 |                11.173508 |
+-----------+----------------------+--------------------------+
|         3 |           153.333333 |               -56.166454 |
+-----------+----------------------+--------------------------+
|         4 |           160.000000 |               -85.342543 |
+-----------+----------------------+--------------------------+
|         5 |           166.666667 |               -67.034528 |
+-----------+----------------------+--------------------------+
|         6 |           173.333333 |               -13.272579 |
+-----------+----------------------+--------------------------+
|         7 |           180.000000 |                45.841172 |
+-----------+----------------------+--------------------------+
|         8 |           186.666667 |                74.387384 |
+-----------+----------------------+--------------------------+
|         9 |           193.333333 |                46.885012 |
+-----------+----------------------+--------------------------+
|        10 |           200.000000 |               -38.787691 |
+-----------+----------------------+--------------------------+


=========================================================
  Root isolation intervals:
=========================================================
  [1] Root is between X = 146.67 and X = 153.333333
  [2] Root is between X = 173.33 and X = 180.000000
  [3] Root is between X = 193.33 and X = 200.000000
```

```
=========================================================
```

Тест 3 (варіант введення 2 зі зменшеним кроком delta = 5 на тому самому відрізку [140; 200], N = ⌊60 / 5⌋ + 1 = 13). Знайдені проміжки стали вужчими:

```
Your choice (1 or 2): 2


Enter start argument x1: 140


Enter start argument x2 (must be greater than X1): 200
Enter step delta (delta > 0): 5


Initial parameter:
X1 = 140.0000, X2 = 200.0000, delta = 5.0000, N = 13


  ... (таблиця з 13 рядків; зупинка після 10-го рядка) ...


=========================================================
  Root isolation intervals:
=========================================================
  [1] Root is between X = 145.00 and X = 150.000000
  [2] Root is between X = 170.00 and X = 175.000000
  [3] Root is between X = 195.00 and X = 200.000000
=========================================================
```

Тест 4 (відрізок, на якому функція не змінює знака).

```
Your choice (1 or 2): 1


Enter start argument x1: 0


Enter start argument x2 (must be greater than X1): 100
Enter number of points N (N => 2): 11


Initial parameter:
X1 = 0.0000, X2 = 100.0000, delta = 10.0000, N = 11


+-----------+----------------------+--------------------------+
|  Point N  |     Argument (X)     |     Function (F(X))      |
+-----------+----------------------+--------------------------+
|         1 |             0.000000 |               450.000000 |
+-----------+----------------------+--------------------------+
|         2 |            10.000000 |               448.387908 |
+-----------+----------------------+--------------------------+
|         3 |            20.000000 |               452.484405 |
+-----------+----------------------+--------------------------+
|         4 |            30.000000 |               429.799850 |
+-----------+----------------------+--------------------------+
|         5 |            40.000000 |               386.536436 |
+-----------+----------------------+--------------------------+
|         6 |            50.000000 |               350.000000 |
+-----------+----------------------+--------------------------+
|         7 |            60.000000 |               329.601703 |
+-----------+----------------------+--------------------------+
|         8 |            70.000000 |               305.078045 |
+-----------+----------------------+--------------------------+
|         9 |            80.000000 |               255.634999 |
+-----------+----------------------+--------------------------+
|        10 |            90.000000 |               193.554790 |
+-----------+----------------------+--------------------------+
Press ENTER to continue
|        11 |           100.000000 |               158.046424 |
+-----------+----------------------+--------------------------+


=========================================================
  Root isolation intervals:
=========================================================
  No sign change detected on the interval [0.00; 100.00].
=========================================================
```

Тест 5 (валідація введення). Програма відхиляє некоректний номер варіанта, X2 ≤ X1, N < 2 та неприпустимий крок delta і повторює запит:

```
Your choice (1 or 2): 3

[ERROR] Invalid option! Please enter 1 or 2.

Your choice (1 or 2): 0

[ERROR] Invalid option! Please enter 1 or 2.

Your choice (1 or 2): 1

Enter start argument x1: 10

Enter start argument x2 (must be greater than X1): 10
[ERROR] X2 must be strictly greater than X1!

Enter start argument x1: 10

Enter start argument x2 (must be greater than X1): 20
Enter number of points N (N => 2): 1
[ERROR] N must be a least 2!
Enter number of points N (N => 2): 0
[ERROR] N must be a least 2!
Enter number of points N (N => 2): 5

Initial parameter:
X1 = 10.0000, X2 = 20.0000, delta = 2.5000, N = 5

Your choice (1 or 2): 2

Enter start argument x1: 0

Enter start argument x2 (must be greater than X1): 50
Enter step delta (delta > 0): 0
[ERROR] Delta must be > 0 and <= (X2 - X1)!
Enter step delta (delta > 0): -5
[ERROR] Delta must be > 0 and <= (X2 - X1)!
Enter step delta (delta > 0): 80
[ERROR] Delta must be > 0 and <= (X2 - X1)!
Enter step delta (delta > 0): 20

Initial parameter:
X1 = 0.0000, X2 = 50.0000, delta = 20.0000, N = 3
```

Для незалежної перевірки знайдених проміжків корені рівняння f(x) = 0 додатково обчислено методом половинного ділення: $\xi_1 \approx 147{,}623132$; $\xi_2 \approx 174{,}742136$; $\xi_3 \approx 197{,}437868$. Кожен корінь лежить усередині одного зі знайдених проміжків у всіх тестах (таблиця 1). Крім того, при $x > 225$ виконується $|(x - 50)\cdot\cos(x/10)| \le x - 50 < 3x - 500$, тому $f(x) < 0$ і додаткових коренів праворуч немає; при $x < 0$ аналогічно $f(x) > 0$.

Таблиця 1 — Проміжки зміни знака функції, знайдені програмою

№	Початкові дані	N	Знайдені проміжки зміни знака
1	X1 = 0, X2 = 300, delta = 20	16	[140; 160], [160; 180], [180; 200]
2	X1 = 140, X2 = 200, N = 10 (delta = 6,6667)	10	[146,67; 153,33], [173,33; 180,00], [193,33; 200,00]
3	X1 = 140, X2 = 200, delta = 5	13	[145; 150], [170; 175], [195; 200]
4	X1 = 0, X2 = 100, N = 11 (delta = 10)	11	змін знака немає

6. Висновки

У результаті виконання другої частини лабораторної роботи № 1 розроблено програму, що розраховує та виводить у вигляді таблиці значення функції $f(x) = (x - 50)\cdot\cos(x/10) - 3x + 500$ (варіант № 16) з двома варіантами введення початкових даних (X1, X2, N або X1, X2, delta), валідацією введених значень (цикли do…while), зупинкою виведення після кожних 10 рядків та програмним визначенням проміжків, на яких функція змінює знак. Отримано практичні навички організації циклів і розгалужень, форматованого виведення (специфікатори %10u, %21.6lf, %25.6lf, що відповідають заданим ширинам стовпців 11, 22 та 26 символів) і побудови блок-схеми алгоритму.

Програму протестовано на кількох наборах даних. Значення функції в контрольних точках збігаються з ручним розрахунком, знайдені проміжки зміни знака містять усі три корені рівняння $f(x) = 0$ ($\xi_1 \approx 147{,}62$; $\xi_2 \approx 174{,}74$; $\xi_3 \approx 197{,}44$), а зменшення кроку з 20 до 5 звужує проміжки ізоляції з 20 до 5 одиниць, що буде використано в лабораторній роботі № 3 як початкове наближення.

Під час розроблення враховано такі особливості: після scanf у вхідному потоці залишається символ нового рядка, тому перед посторінковим виведенням додано виклик getchar(); при заданому кроці delta кількість точок визначається цілою частиною відношення (X2 − X1) / delta, тому якщо крок не ділить відрізок націло, останнє значення аргументу буде меншим за X2; критерій $f(x_{i-1})\cdot f(x_i) \le 0$ виявляє лише непарну кількість коренів на проміжку, тому для надійного знаходження близько розташованих коренів крок слід

обирати достатньо малим; масиви меж проміжків мають розмір 100, чого для даної функції достатньо (коренів лише три), але для інших функцій потрібна перевірка виходу за межі масиву.